Lab 06 – Python List

Programming Exercise:

1. Take a sample list `[2, 1, 3, 5, 4, 3, 8]. Apply` del(), remove(), sort(), insert(), pop(), extend()…)

Source Code:

```
my_list = [2, 1, 3, 5, 4, 3, 8]
# 1. Using del() to delete an element by index
del(my_list[2])  # Deletes the element at index 2 (which is 3)
print("After del():", my_list)
# 2. Using remove() to remove an element by value (first occurrence)
my_list.remove(3)  # Removes the first occurrence of 3
print("After remove():", my_list)
# 3. Using sort() to sort the list in ascending order
my_list.sort()  # Sorts the list in ascending order
print("After sort():", my_list)
# 4. Using insert() to insert an element at a specific position
my_list.insert( __index: 2,  __object: 7)  # Inserts 7 at index 2
print("After insert():", my_list)
# 5. Using pop() to remove and return an element by index
popped_element = my_list.pop(4)  # Removes and returns the element at index 4
print("After pop():", my_list)
print("Popped element:", popped_element)
# 6. Using extend() to add multiple elements from another list
my_list.extend([10, 12, 14])  # Adds elements 10, 12, and 14 to the list
print("After extend():", my_list
```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Scripts
After del(): [2, 1, 5, 4, 3, 8]
After remove(): [2, 1, 5, 4, 8]
After sort(): [1, 2, 4, 5, 8]
After insert(): [1, 2, 7, 4, 5, 8]
After pop(): [1, 2, 7, 4, 8]
Popped element: 5
After extend(): [1, 2, 7, 4, 8, 10, 12, 14]

Process finished with exit code 0
```

2. A ladder put up right against a wall will fall over unless put up at a certain angle less than 90 degrees. Given variables length and angle storing the length of the ladder and the angle that it forms with the ground as it leans against the wall, write a Python expression involving length and angle that computes the height reached by the ladder. Evaluate the expression for these values of length and angle:

(a)16 feet and 75 degrees
(b)20 feet and 0 degrees
(c)24 feet and 45 degrees
(d)24 feet and 80 degrees

Source Code:

```
import math
# Function to calculate height
def calculate_height(length, angle_degrees):  4 usages
    angle_radians = math.pi * angle_degrees / 180  # Convert degrees to radians
    height = length * math.sin(angle_radians)  # Calculate height
    return height

print("Height for (a) 16 feet and 75 degrees:", calculate_height( length: 16,  angle_degrees: 75))
print("Height for (b) 20 feet and 0 degrees:", calculate_height( length: 20,  angle_degrees: 0))
print("Height for (c) 24 feet and 45 degrees:", calculate_height( length: 24,  angle_degrees: 45))
print("Height for (d) 24 feet and 80 degrees:", calculate_height( length: 24,  angle_degrees: 80))

```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Scripts\python.exe "C:\Users\hp\Pych
Height for (a) 16 feet and 75 degrees: 15.454813220625093
Height for (b) 20 feet and 0 degrees: 0.0
Height for (c) 24 feet and 45 degrees: 16.970562748477143
Height for (d) 24 feet and 80 degrees: 23.63538607229299

Process finished with exit code 0
```

3. Write the relevant Python expression or statement, involving a list of numbers lst and using list operators and methods for these specifications:

(a)An expression that evaluates to the index of the middle element of lst

(a) `middle_index = len(lst) // 2`

(b)An expression that evaluates to the middle element of lst

(b) `middle_element = lst[len(lst) // 2]`

(c)A statement that sorts the list lst in descending order

(c) `lst.sort(reverse=True)`

(d)A statement that removes the first number of list lst and puts it at the end

(d) `lst.append(lst.pop(0))`

Note: If a list has even length, then the middle element is defined to be the rightmost of the two elements in the middle of the list.

4. Start by assigning to variables monthsL and monthsT a list and a tuple, respectively, both containing strings 'Jan', 'Feb', 'Mar', and 'May', in that order. Then attempt the following with both containers:

(a) Insert string 'Apr' between 'Mar' and 'May'.
(b)Append string 'Jun'.
(c)Pop the container.
(d)Remove the second item in the container.
(e)Reverse the order of items in the container.
(f)Sort the container.

Note: when attempting these on tuple monthsT you should expect errors.

Source Code:

```
# Create a list and a tuple with the same elements
monthsL = ['Jan', 'Feb', 'Mar', 'May']
monthsT = ('Jan', 'Feb', 'Mar', 'May')

#(a)
    # FOR LIST
monthsL.insert( __index: 3,  __object: 'Apr')
print("After inserting 'Apr':", monthsL)
   # FOR TUPLE
# This will raise an error
# monthsT.insert(3, 'Apr')

#(b)
   # FOR LIST
monthsL.append('Jun')
print("After appending 'Jun':", monthsL)
   # FOR TUPLE
# This will raise an error
# monthsT.append('Jun')

#(c)
   # FOR LIST
last_item = monthsL.pop()
print("After popping:", monthsL)
print("Popped item:", last_item)
   # FOR TUPLE
# This will raise an error
# monthsT.pop()

```

```python
#(d)
   # FOR LIST
del monthsL[1]  # Alternatively, monthsL.pop(1)
print("After removing the second item:", monthsL)
   # FOR TUPLE
# This will raise an error
# del monthsT[1]

#(e)
   # FOR LIST
monthsL.reverse()
print("After reversing:", monthsL)
   # FOR TUPLE
# This will raise an error
# monthsT.reverse()

#(f)
   # FOR LIST
monthsL.sort()
print("After sorting:", monthsL)
   # FOR TUPLE
# This will raise an error
# monthsT.sort()

```

Output:

```
C:\Users\hp\PycharmProjects\main\venv\Scripts\python.exe "C:\Users\hp\
After inserting 'Apr': ['Jan', 'Feb', 'Mar', 'Apr', 'May']
After appending 'Jun': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
After popping: ['Jan', 'Feb', 'Mar', 'Apr', 'May']
Popped item: Jun
After removing the second item: ['Jan', 'Mar', 'Apr', 'May']
After reversing: ['May', 'Apr', 'Mar', 'Jan']
After sorting: ['Apr', 'Jan', 'Mar', 'May']

Process finished with exit code 0
```

6. Write the corresponding Python assignment statements:

(a)Assign 6 to variable a and 7 to variable b.

(b)Assign to variable c the average of variables a and b.

(c)Assign to variable inventory the list containing strings 'paper', 'staples', and 'pencils'.

(d)Assign to variables first, middle and last the strings 'John', 'Fitzgerald', and 'Kennedy'.

(e)Assign to variable fullname the concatenation of string variables first, middle, and last. Make sure you incorporate blank spaces appropriately.

```
# Part (a)
a = 6
b = 7

# Part (b)
c = (a + b) / 2

# Part (c)
inventory = ['paper', 'staples', 'pencils']

# Part (d)
first = 'John'
middle = 'Fitzgerald'
last = 'Kennedy'

# Part (e)
fullname = first + ' ' + middle + ' ' + last
```